Global Research & Survey App (GRS) — Technical Blueprint

First use: Door-to-Door Skills Campaign (forms DD/S/001/2026 and DD/S/002/2026) · Android first, iOS later · Flutter + NestJS + Supabase · Offline-first

Prepared for Imantha · Version 1.3 · 25 October 2026

1. Purpose and Goals

The **Global Research & Survey App (GRS)** lets field teams collect survey data on an Android phone and keeps every record in **one central database**, even when the agent has no internet. Its first use is the Door-to-Door Skills Campaign, where teams visit households and record each person's education, vocational background, interests and welfare benefits. Today that work is done on paper forms, and this app replaces them.

Goals

- Agents fill in household and client details on an Android phone.
- Everything works **offline**; data is saved on the phone and synced later.
- All data ends up in one Supabase database that officers at every level can view according to their role.
- Simple to build, simple to use, easy to extend to iOS.

Non-goals for version 1 (to keep it simple): photo or document uploads, in-app chat, push notifications, analytics beyond basic counts, complex approval workflows.

2. Decisions Already Made

Topic	Decision
App name	Global Research & Survey App (GRS)
Mobile framework	Flutter (one codebase)
Backend API	NestJS (Node.js, TypeScript)
Database	Supabase (managed PostgreSQL)
First platform	Android; iOS later from the same code

Topic	Decision
Mobile users	Levels 0 to 5 (Level 00 works on the web), on their own phones (Android 10 or newer)
Web users	All levels, Level 00 first
Expected agents	Long-term goal of 50,000 to 1,000,000; the first launch is smaller
Offline sync	Automatic when internet returns **and** a manual "Sync now" button
Languages	Sinhala, English and Tamil in the app, dashboard and reports
Accounts	Created by admins; generated ID (GRS + 10 digits) and random password; users can switch to email login and change the password
House list loading	Bulk import from Excel, or typed one by one
Reports	The platform's own reports (not a government format)
Distribution	Direct APK for now
Data access	Super admin and admin Levels 0 to 4, each seeing the records related to them

Verdict on your stack: Flutter + NestJS + Supabase is still a good fit, but at the long-term goal of 50,000 to 1,000,000 agents the design around it must change: a larger database, careful sync traffic, pre-calculated dashboard numbers and load testing (section 6.1). The stack itself does not have to change.

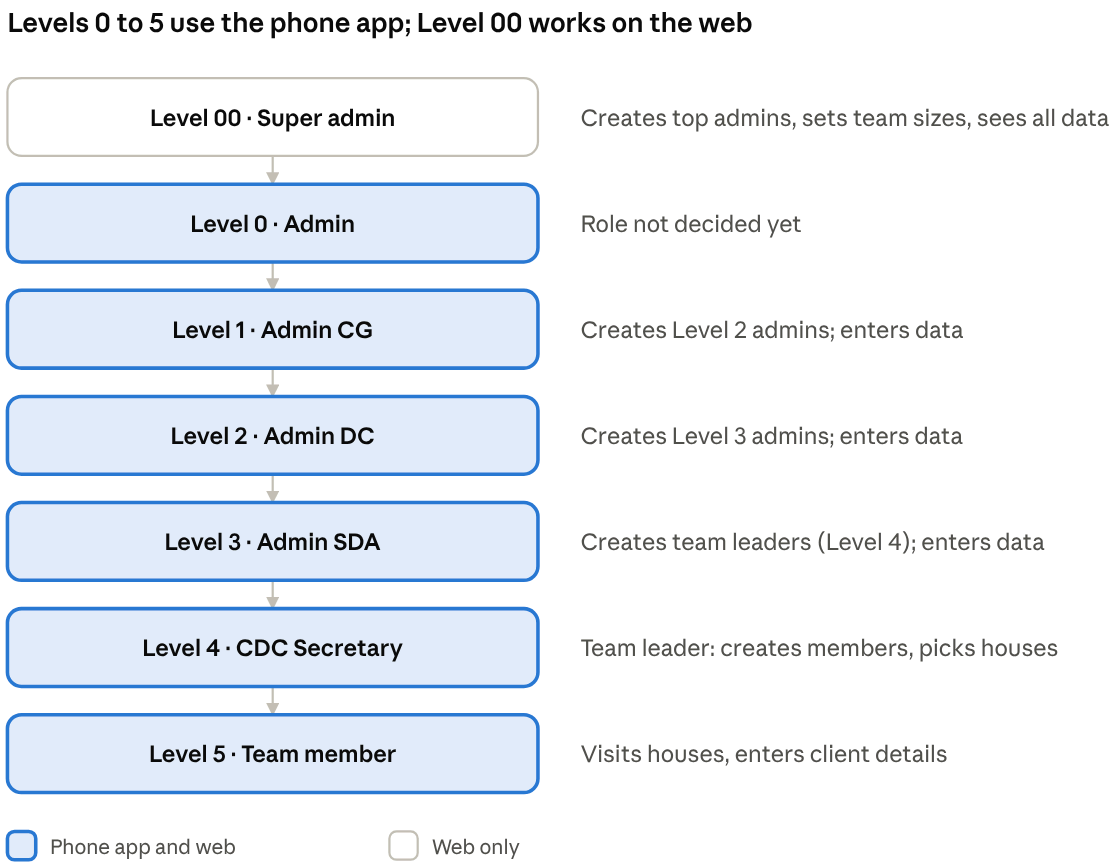
3. Users and Roles

The hierarchy comes from the DD/S/002/2026 admin panel design. Each level sees only the data below it.

Level	Role	Platform	Main job
00	Super admin	Web	Full system control; sets team member limits; sees all data
0	Admin (role not decided yet)	Web and mobile	To be decided with the client
1	Admin CG	Web and mobile	Creates Level 2 admins and own team members; enters data; sees the records of their area
2	Admin DC	Web and mobile	Creates Level 3 admins and own team members; enters data; sees the records of their area
3	Admin SDA	Web and mobile	Creates Level 4 team leaders and own team members; enters data; sees the records of their area
4	CDC Secretary (team leader)	Mobile (and web)	Creates Level 5 members for their team; chooses the team's houses; enters data; sees the team's records
5	CDC Team member	Mobile	Visits houses and enters client details; sees their own records

Build order: Level 00, Level 4 and Level 5 are built first. Levels 1 to 3 use the same app and dashboard, plus account creation, and follow next. Level 0 waits until its role is decided.

Account creation: every account is created by an admin on the level above, and the system generates the ID and password (section 3.1). Section 3.2 shows who creates whom.



account hierarchy · 7 levels

Every level from 0 to 5 can use the Android app and enter data; Level 00 works on the web.

3.1 Accounts: generated ID and password

Every account is created by an admin on the level above. The system generates the login details, so nobody chooses or shares a password by hand.

1. The admin creates the account (name, role, area, team), one at a time or in bulk. The system generates an **ID** made of GRS and 10 random digits (for example GRS1234567890) and a random **password**.
2. The admin sees the ID and password once and hands them to the person: a printed sheet, or a downloaded list for bulk creation.
3. The person signs in with the ID and password. The first sign-in needs internet and forces a new password.
4. The person can add their own email, verified by a code. From then on they can sign in with either the ID or the email, plus the password.

5. Forgotten password: if an email was added, it is reset by an emailed code; otherwise the admin who created the account issues a new random password.

Rules that keep this safe

- IDs are random, not sequential, and checked for duplicates when the account is created.
- A 10-digit ID is easy to guess in principle, so login errors never say whether an ID exists, and logins are rate-limited per ID, email and device.
- Passwords are random, stored only as hashes (Argon2), and cannot be shown again after the admin sees them once; a reset makes a new one.
- A password that is never used expires after a set time.
- A deactivated account is refused at once, even on a phone that is already signed in.
- The team's member limit is checked when an admin creates a member; a full team cannot take more.
- One email can belong to one account only.

3.2 Who creates whom

Admins create the accounts for the level directly below them, and Levels 1 to 4 also create their own team members. This is my reading of the client's note and should be confirmed.

Level	Can create accounts for
00	Level 0 and Level 1 admins, and any other level
0	To be decided
1	Level 2 admins, and their own team members
2	Level 3 admins, and their own team members
3	Level 4 team leaders, and their own team members
4	Level 5 team members of their team
5	No one

A team's member limit, set by Level 00 (or Level 0 later), stops an admin from creating more members than the team allows.

3.3 What each level can see and enter

Level	App	Enters data	Sees records of
00	Web	Not stated	All records
0	Web and mobile	Yes	Own area and below, plus own entries (role to be decided)
1 to 3	Web and mobile	Yes	Own area and below, plus own entries
4	Mobile (and web)	Yes	Own team and below, plus own entries
5	Mobile	Yes	Own entries

Every level has a records view that shows the records related to it.

4. What Data Is Collected

4.1 Pre-visit data (DD/S/002/2026), entered by Level 4

- District, Divisional Secretariat, GN division number and name
- CDC Secretary name, contact number, email
- House list taken from the **election name list**: serial number (auto), house number, house leader name, address, village name
- Beneficiary flags per house: EPF, ETF, Aswesuma, Samurdhi, special needs
- Number of target beneficiaries, age group (for example 18 to 25), language used (Sinhala, Tamil, English)
- Team grouping: which house numbers belong to which team (Team 1 gets houses 25, 26, 28 and so on)

4.2 Client data (DD/S/001/2026), entered by Level 5 at the door

House-level fields from section 4.1 appear automatically; the agent fills in:

1. Name
2. Age
3. Gender (female or male)

4. Address
5. Phone number
6. WhatsApp number
7. Highest education qualification
8. Highest vocational qualification
9. Interested vocational stream
10. Interested job
11. Beneficiary type (EPF, ETF, Aswesuma, Samurdhi)
12. Special needs: yes or no
13. Future hopes (one or more): higher education, vocational education, entrepreneur, local job, foreign job
14. Information provided by: self, mother, father, sister, other

One house can have **several clients**, so the app loops "Enter next beneficiary" until the agent taps End.

4.3 Loading the house list: Excel import or typing

Level 4 can load the houses from the election name list in two ways.

- **Excel import (for large lists):** download the template, fill it in (house number, house leader, address, village, beneficiary flags, target counts), and upload it on the web dashboard. The API checks every row and shows a preview with errors per row, such as a missing house number or a duplicate in the same GN division. Nothing is saved until the secretary confirms.
- **Typed entry:** one house at a time in the app or the dashboard, with the serial number added automatically.
- Each import is recorded as a batch (who, when, how many rows passed and failed).
- Version 1 keeps Excel import on the web dashboard, which Level 4 can also use, so the phone app stays simple. Each file must stay under 4.5 MB because of the hosting limit, which should be ample for a GN division's house list.

Excel template columns (one row per house), taken from the Level 4 section of the format file:

Column	Required	Notes
House No	Yes	Unique inside the GN division

Column	Required	Notes
Name of house leader	Yes	As on the election name list
Address	No	
Name of village	No	
Aswesuma	No	Yes or no
ETF	No	Yes or no
EPF	No	Yes or no
Samurdhi	No	Yes or no
Special needs	No	Yes or no
Target beneficiaries: female	No	Whole number
Target beneficiaries: male	No	Whole number
Age group	No	Such as 18-25
Language: Sinhala	No	Yes or no
Language: Tamil	No	Yes or no
Language: English	No	Yes or no

The serial number is added automatically. District, divisional secretariat, GN division number and name, and the CDC Secretary's name, contact and email are set once for the GN division, so they are not repeated in every row.

5. Technology Stack

Layer	Technology	Why
Mobile app	**Flutter** (Dart)	One codebase for Android now, iOS later, plus web
State management	**Riverpod**	Simple, popular, testable

Layer	Technology	Why
Local database (offline)	**Drift** (SQLite) with SQLCipher	Reliable structured storage on the phone, encrypted
Navigation	go_router	Standard Flutter routing
Connectivity detection	connectivity_plus + internet_connection_checker_plus	Know when internet is really back
Background sync	workmanager	Runs sync on Android even if the app is closed
Backend API	**NestJS** (Node.js, TypeScript)	One API for the app and the dashboard: login, sync, import, reports, role checks
Data access in the API	Prisma (or TypeORM)	Typed queries and database migrations
Database	**Supabase** (managed PostgreSQL)	Backups, dashboard, familiar SQL
Authentication	JWT access and refresh tokens issued by NestJS; passwords hashed with Argon2	Supports login by ID or email and generated passwords
Validation and API docs	class-validator and Swagger (OpenAPI)	Same rules for every client, clear contract
Excel import and export	ExcelJS	Reads the house-list template, writes Excel reports
PDF reports	A PDF library, chosen after a Sinhala and Tamil font test	Sinhala and Tamil text must render correctly
Email	A transactional email service	Verification and password-reset codes
Web dashboard	**Flutter Web** (same codebase and models)	Avoids learning a second framework

Layer	Technology	Why
Localization	Flutter intl (ARB files), Noto Sans Sinhala and Noto Sans Tamil fonts	Sinhala, English and Tamil on every phone
API hosting	Vercel (likely), as a serverless function	Limits to design around are in section 6
Error tracking	Sentry (free tier)	See crashes from field devices
Source control and CI	GitHub + GitHub Actions	Automated tests and signed Android builds

Why not a bigger sync framework? Tools like PowerSync give automatic two-way sync, but add cost and complexity. Your data is mostly one-way (agent to server) and each house is handled by one team, so a small custom "outbox" sync is enough. You can upgrade later if needs grow.

5.1 Languages: Sinhala, English and Tamil

- A language picker on the login screen and in settings; the choice is saved on the profile and applies to the app, the dashboard and exports.
- Every screen, button, validation message and error is translated, with one translation file per language.
- Dropdown values (education, vocational stream, jobs, future hopes) are stored as stable codes with three labels, so any report can show any language.
- Noto Sans Sinhala and Noto Sans Tamil are bundled in the app so text looks the same on every phone, including older ones.
- Names and addresses can be typed in any of the three scripts; text is stored as UTF-8 end to end.
- Sinhala and Tamil wording is reviewed by native speakers before the pilot, using the wording of the official forms.

6. System Architecture

The system has four parts: the Android app used in the field, the NestJS API, the Supabase database, and the web dashboard used by higher officers. Both apps talk only to the API.

- **Android app (Levels 0 to 5):** UI → Riverpod → Repository → Drift local DB → Sync Service → NestJS API.
- **NestJS API:** modules for login and accounts, sync, Excel import, reports, and user and area management, with role guards that limit every request to the user's area.
- **Supabase (PostgreSQL):** stores all data; Row Level Security stays on as a second safety net.
- **Web dashboard (all levels, Level 00 first):** Flutter Web calling the same API.

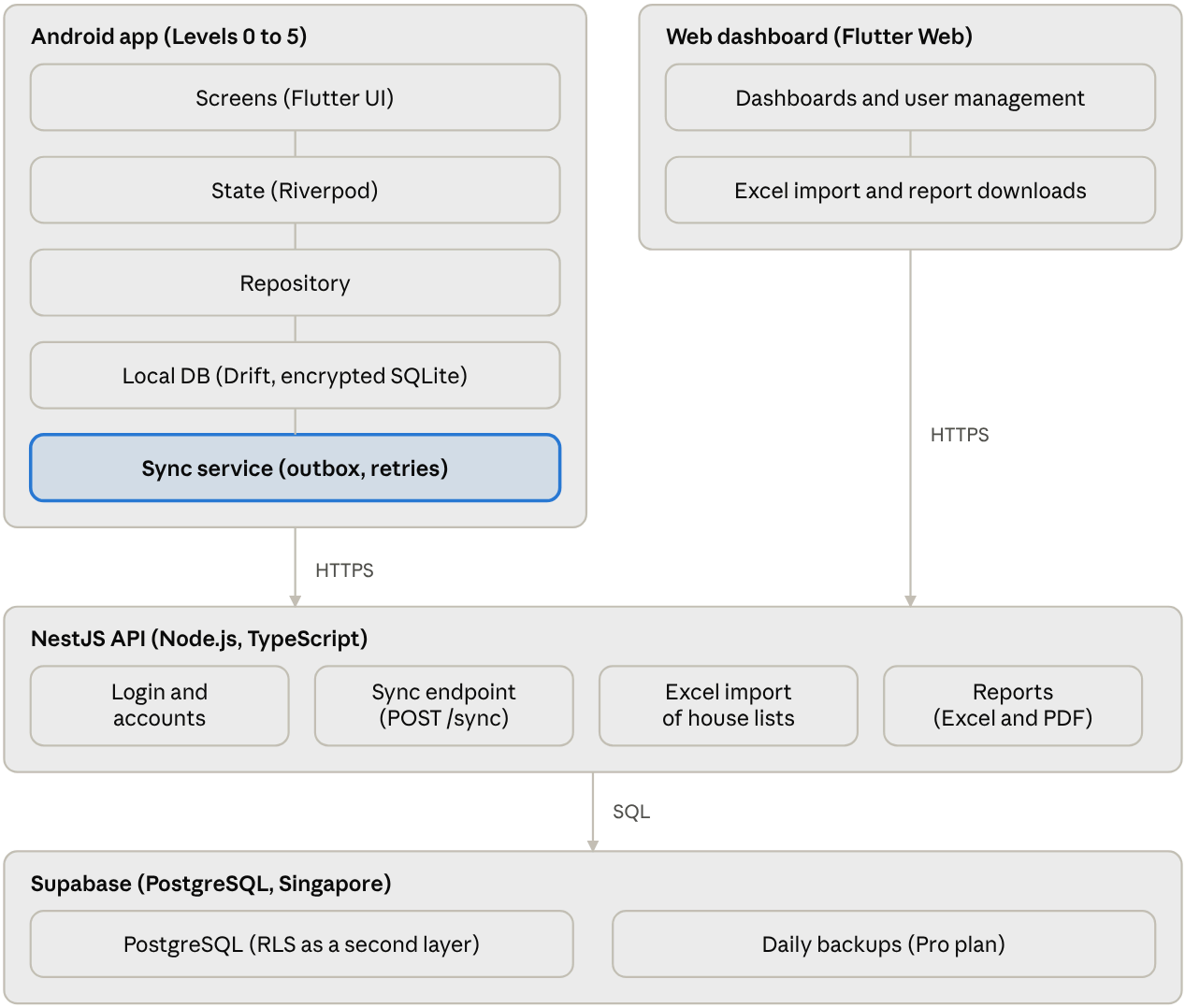
Why an API in the middle: the account rules, Excel import, sync checks and reports live in one place, and the apps never hold database keys.

Hosting recommendation: Supabase, region **Singapore** (closest to Sri Lanka), on a paid plan with a compute size chosen after load tests (section 6.1). The free tier pauses inactive projects and has no daily backups, which is not acceptable for real citizen data.

NestJS on Vercel (your likely choice): Vercel runs a NestJS app as one serverless function, which suits this project if the limits in Vercel's function limits are designed around from the start:

- **Region:** functions run in one region, `iad1` (US East), by default. Change it to the region nearest the Singapore database, or every request pays a long trip.
- **Request and response size:** 4.5 MB each. Sync batches are small text and fit; Excel imports and report downloads must stay under it, or go through Supabase Storage with a link.
- **Run time:** 300 seconds by default (up to 800 on the Pro plan), enough for a large import or report.
- **Bundle size:** 250 MB uncompressed, so PDF reports should use a light library, not a bundled browser.
- **Database connections:** connect through Supabase's connection pooler, because each function opens short-lived connections and Vercel limits open file descriptors to 1,024, shared across running functions.
- **No background loop:** nothing runs between requests, so scheduled jobs use Vercel Cron instead of a scheduler inside NestJS.
- **Plan:** check that the free Hobby plan's terms allow a government campaign; the Pro plan raises the limits above.
- **Cost at scale:** usage-based charges are likely to grow with the number of requests and the data moved, which is another reason to batch sync requests (section 6.1).

Both apps talk to one NestJS API, which alone writes to the database



architecture · 2 apps, NestJS API, Supabase

Both apps reach the data only through the NestJS API; the phone does so through its sync service.

6.1 Scaling for 50,000 to 1,000,000 agents

50,000 to 1,000,000 agents is the long-term goal, not the first launch, so the plan is to build the habits below in from day one and grow the database size in steps. The stack itself stays the same.

Keep sync traffic low

- The app syncs only when records are waiting, never on a fixed timer, and every phone adds a random delay so thousands of phones do not call the server in the same second.
- Records travel in batches (for example 50 at a time), and a record saved while online joins the next small batch instead of making its own request.
- When the server is busy it answers "slow down" with a wait time (HTTP 429), and the app backs off with a growing, randomised delay.

Keep the database fast

- Pick a larger Supabase compute size than the default, test it, and resize after the pilot. Connect through the pooler.
- Index the area columns (district, divisional secretariat, GN division, team) and the user and date columns. Every list is filtered by area and paged by position (keyset), never by offset.
- Save each batch with one bulk upsert, not one query per record.
- When the clients table reaches tens of millions of rows, split it into partitions (by district or by month).
- Keep rows narrow, with long text kept out of the busiest tables.

Keep dashboards cheap

- Totals and progress come from summary tables, updated as records arrive or by a scheduled job (Vercel Cron), not by counting millions of rows on every page load.
- Record tables are always area-filtered and paged.
- Exports and reports run as background jobs that write a file to Supabase Storage and give the admin a download link; they never stream a big file through the API.

Keep accounts manageable

- Admins create accounts in bulk (for example "create 25 team members") and download a credential sheet.
- Login lookups use indexes on the ID and the email, with rate limits per ID, email and device.
- Verification and password-reset emails at this volume need a transactional email plan sized for it.

Prove it before the rollout

- Load test with simulated agents (for example thousands syncing during the 10 a.m. to 3 p.m. window) before the pilot ends, and again before each large district is added.
- Roll out district by district, watching database load and error rates.

- Supabase and Vercel costs grow with usage; get a first estimate from the pilot numbers.

From the client: the busiest hours are most probably 10 a.m. to 3 p.m., so load tests simulate that window. How many records an agent enters per day depends on the campaign and is still unknown, so the test targets stay adjustable.

7. Offline-First Design

This is the most important part of the app. The principle: **the phone's local database is the source of truth while working; the server is updated afterwards.**

7.1 How saving works

1. The agent fills in the form and taps Save.
2. The record is written to the local SQLite database with `sync_status = pending` and a **client-generated UUID**.
3. The agent immediately sees it in their list with a "Waiting to sync" badge. No internet needed.

7.2 How syncing works

Sync starts in three ways:

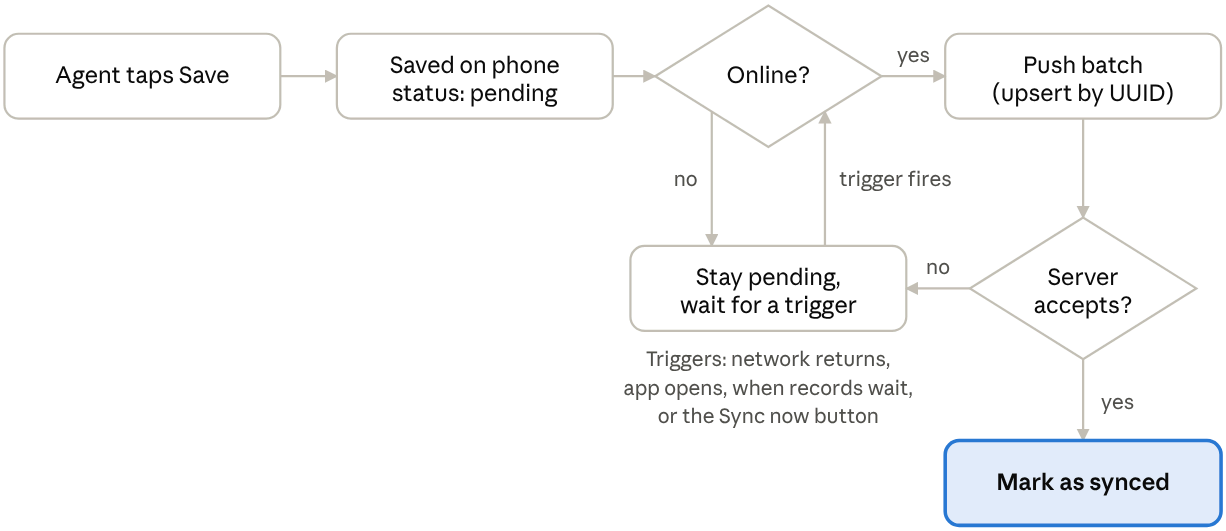
- **Automatic:** when connectivity returns, when the app opens, and in the background **only while records are waiting**, after a random delay so thousands of phones do not call the server at the same moment.
- **Manual:** the "Sync now" button, with a counter such as "12 records waiting".
- **After save:** if online, the record joins the next small batch a few seconds later instead of making its own request.

The sync service then:

1. Collects all records where `sync_status = pending` or `failed`.
2. Sends them to the NestJS API (`POST /sync`) in batches (for example 50 at a time); the API checks the user's role and area, then saves them using **upsert by UUID**.
3. On success, marks them `synced` and stores the server time.
4. On failure, keeps them `pending`, increases `retry_count`, and retries later with a growing, randomised delay.

Records stay on the phone until the server confirms them

A failed or offline attempt just leaves the record pending, so nothing is lost



sync flow · 2 decisions, 1 retry loop

Android field data app with offline syncA failed or offline attempt leaves the record pending; the next trigger tries again.

7.3 Why duplicates cannot happen

Because every record has a UUID created on the phone, sending the same record twice simply updates the same row. A dropped connection halfway through a batch is safe.

7.4 Conflict rule (kept simple)

- A house is assigned to one team, so two agents rarely edit the same client.
- If it does happen, **last edit wins** using the `updated_at` timestamp.
- Records already `synced` can still be edited; the edit becomes `pending` again.

7.5 Data the phone downloads

Before going to the field, the agent taps **"Download my area"** (needs internet once). The phone stores:

- The team's house list and their pre-filled house details
- Dropdown values (education levels, vocational streams, jobs)
- The user's profile and assigned team

Level 5 can then work for days offline.

7.6 Login offline

The app keeps the login tokens in Android secure storage. The first sign-in and the first-time account setup need internet; after that the agent can open the app offline. If the token has expired and there is no internet, the app shows a clear message and still lets the agent keep recording data, which waits on the phone until the agent signs in again and sync can run.

7.7 Sync status shown to the user

Status	Meaning	Badge
Draft	Not finished	Grey
Pending	Saved on phone, waiting to sync	Orange
Synced	Safely on the server	Green
Failed	Tried and failed; will retry	Red

8. Database Design (Supabase / PostgreSQL)

8.1 Tables

Table	Key columns
`profiles`	id, user_code (GRS + 10 digits), email, email_verified, password_hash, must_change_password, full_name, phone, preferred_language, role_level (0 to 5; 00 stored as -1 or a flag), created_by, district_id, ds_id, gn_id, team_id, is_active
`districts`	id, name_si, name_ta, name_en
`divisional_secretariats`	id, district_id, name
`gn_divisions`	id, ds_id, gn_number, gn_name, cdc_secretary_id, secretary_contact, secretary_email

Table	Key columns
`teams`	id, gn_id, team_no, leader_id, max_members
`households`	id (UUID), gn_id, team_id, serial_no, house_no, house_leader_name, address, village_name, is_epf, is_etf, is_aswesuma, is_samurdhi, has_special_needs, target_female, target_male, age_group, lang_sinhala, lang_tamil, lang_english, visit_status, created_by, updated_at
`clients`	id (UUID), household_id, name, age, gender, address, phone, whatsapp, highest_education, highest_vocational, interested_vocational_stream, interested_job, is_epf, is_etf, is_aswesuma, is_samurdhi, has_special_needs, future_hopes (array), info_provided_by, collected_by, collected_at, device_id, client_updated_at, synced_at
`lookup_values`	id, category (education, vocational, job), code, label_si, label_ta, label_en
`import_batches`	id, gn_id, file_name, rows_total, rows_ok, rows_failed, imported_by, imported_at
`app_releases`	version, min_supported_version, apk_url, checksum, released_at
`sync_log` (optional)	id, user_id, device_id, records_sent, status, created_at

8.2 Relationships

District → Divisional Secretariat → GN Division → Team → House → Client. Every table carries the area ids so security rules stay simple.

Every client sits under a house, a team, a GN division and a district



profiles (users) link to a GN division and a team; lookup_values feeds the dropdowns.

table relationships · 6 tables

Each table nests inside the one before it, so security rules can filter every row by area.

9. Security and Privacy

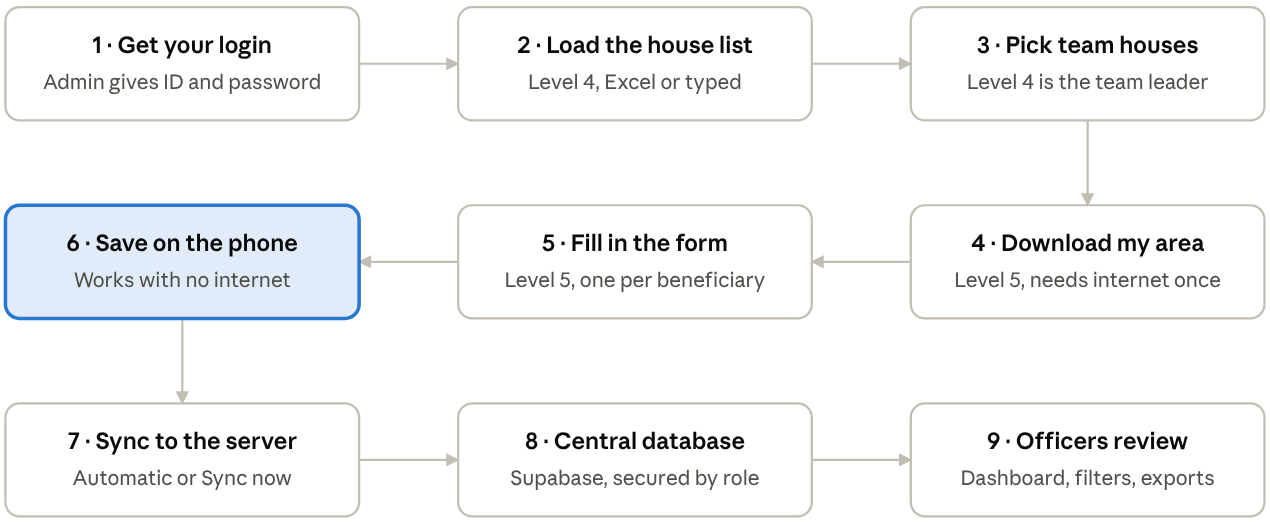
This app collects personal data of citizens, including phone numbers and welfare status, so security is part of the design, not an extra.

- **Authorization in the API:** every request passes role guards and an area filter, so Level 5 sees only their own entries, Level 4 their team, and each higher level its own area and below, plus the records they entered themselves. Row Level Security in Supabase stays on as a second layer.
- **No direct database access from the apps:** only the NestJS API connects to Supabase; the apps never hold database keys.
- **Generated IDs and passwords:** IDs and passwords are random, passwords are hashed with Argon2 and shown once to the admin who creates them, a new password is forced at first sign-in, login errors never reveal whether an ID exists, and logins are rate-limited with lockout after repeated wrong passwords.
- **Encrypted transport and storage:** HTTPS everywhere; Supabase encrypts data at rest.
- **Agents' own phones:** the local SQLite database is encrypted with SQLCipher, the key sits in Android Keystore, and the app can ask for a PIN or fingerprint. The app warns before logout if records are unsynced.

- **Lost or shared phones:** an admin deactivates the account and the API rejects that phone's tokens immediately.
- **Data access:** the super admin and admin Levels 0 to 4 can view and export the records of their own area and below; Level 5 sees only their own entries.
- **Consent:** the app shows a short consent statement before the agent records someone's details, and stores that consent was given.
- **Sri Lanka PDPA:** confirm who the data controller is and who may access exports; no deletion period has been set yet.
- **APK integrity:** every release is signed with one protected key and published with a checksum; the app checks its version at startup.
- **Backups:** daily automatic backups (Supabase paid plan) plus a monthly export kept elsewhere.
- **Audit:** `created_by`, `updated_at`, and device id on every record.

10. App Screens

Nine steps take a record from the door to the dashboard



end-to-end flow · 9 steps

Steps 1 to 3 prepare the accounts and the area; steps 4 to 9 repeat as agents visit houses.

10.1 Android app: Level 5 (Team member)

1. First sign-in: log in with the ID and password from your admin, choose a new password, and add an email (recommended, because it allows password reset by email)
2. Login with ID or email, plus password
3. Home: team name, houses assigned, houses done, records waiting to sync, **Sync now** button, online/offline indicator
4. House list: searchable list with status (not visited, partly done, done)
5. House detail: pre-filled info (house number, leader, address, beneficiary flags)
6. Client form (sections 10.1 to 10.13 of the form), with "Save and add next beneficiary" and "Save and finish house"
7. My records: everything I entered, with pending and synced status and an edit option
8. Settings: language (Sinhala, Tamil, English), download my area, logout

10.2 Android app: Level 4 (CDC Secretary), in addition

1. GN division setup (secretary details)
2. Add houses by typing, one at a time (serial number auto-generated); bulk Excel import is on the web dashboard
3. Choose the houses for your team from the GN division's list; a house can belong to one team only, and houses taken by another team show as taken
4. Create team members: the ID and password are generated and shown once; the admin sets the team's member limit, and a full team cannot take more members
5. Team progress overview
6. Team records: view the records entered by the team

10.3 Android app: Levels 0 to 3

1. Create accounts for the level below and for your own team members, one at a time or in bulk, with a credential sheet to share
2. Enter data with the same client form as Level 5
3. View the records related to your level: your own entries and your area's records, with filters and search
4. Progress overview for your area

10.4 Web dashboard: all levels, Level 00 first

1. Login with ID or email
2. Dashboard: totals and progress by district, division, GN division and team; each level sees its own area, and the numbers are pre-calculated
3. User management: create accounts for the level below (one at a time or in bulk, with a credential sheet to download), deactivate accounts, reset passwords
4. Area and team management: districts, divisions, GN divisions, teams, and a member limit for each team (set by Level 00 for now)
5. Excel import of house lists, with a preview and an error report per row
6. Records: tables of the records related to the signed-in level (own area and below, plus own entries), with filters (area, language, beneficiary type) and **export to Excel/CSV**
7. Reports (Excel and PDF) in Sinhala, English or Tamil
8. Data entry for admins, with the same client form as the app
9. Sync health: which teams have unsynced data (optional)

11. Project Phases and Timeline

Estimates are for a small team of one to two developers and are rough; adjust after the pilot.

Phase	What gets built	Output	Estimate
0. Finalise requirements	Confirm both forms, dropdown values, role rules, Excel template columns, our own report layouts, data retention	Signed-off field list, roles and report formats	1 week

Phase	What gets built	Output	Estimate
1. Backend foundation	NestJS project, Supabase database, tables, login by ID or email, generated IDs and passwords, forced password change, role guards, seed districts and lookups, test accounts per level, a Vercel test deployment (region, import, reports), indexes and summary tables for scale	Working secure API	2 to 3 weeks
2. Android app: Level 5 core	Login and first-time password change, language switch, download my area, house list, client form, local Drift DB, save offline	Agent can record data with no internet	3 weeks
3. Sync engine	Outbox, sync endpoint, auto and manual sync, retry, background task, sync status UI	Data safely reaches the database	2 weeks
4. Level 4 features	Typed house entry, choosing houses for a team, creating team members, team progress overview	Secretary can prepare the whole GN division	2 weeks
5. Web dashboard	User management with bulk account creation, team member limits, Excel house-list import, data tables, progress (Level 00 first)	Level 00 can manage, monitor and export	3 weeks
6. Reports	Excel and PDF reports designed for this platform, report language choice	Reports approved by the client	1 to 2 weeks
7. Languages and polish	Native-speaker review of Sinhala, English and Tamil, fonts, validation, error messages	Field-ready UX	1 to 2 weeks

Phase	What gets built	Output	Estimate
8. Pilot	One GN division, 5 to 10 agents on their own phones, direct APK install, a load test with simulated agents, fix issues	Pilot report	2 weeks
9. Rollout	Training material, signed APK download page, in-app update check, support	Live campaign	2 weeks
10. iOS (later)	Build and test the same Flutter code on iOS, App Store release	iOS app	2 to 3 weeks

Total to Android launch (Levels 00, 4 and 5): roughly **21 to 24 weeks**, including about 2 weeks for scaling work and load testing. Phases 1 to 4 deliver the core value; the dashboard can start smaller if officers first rely on exports, and the report layouts need client sign-off in Phase 0. Levels 1 to 3, with account creation, data entry and their own records views, follow the first launch and are not in this estimate; Level 0 waits until its role is decided.

12. Testing Plan

- **Offline scenarios:** airplane mode, saving 100 records, reconnecting; killing the app mid-sync; switching networks mid-sync.
- **Duplicate check:** sync the same record twice and confirm one row.
- **Security:** log in as each level and confirm no records outside their area are visible, and that each level's records view shows only what is related to it.
- **Accounts:** a generated ID and password work for the first sign-in, a new password is forced, an email can be added and then both the ID and the email log in, a full team refuses new members, two team leaders cannot claim the same house, and a deactivated account is refused at once.
- **Excel import:** missing columns, blank rows, duplicate house numbers, and Sinhala and Tamil text in the file.
- **Reports:** check each report against the layout the client approved, in all three languages.

- **Own phones:** several phone brands, including 2 to 3 GB RAM models and Android 10 as the lowest version; install the APK directly and update from an older version.
- **Language:** Sinhala, Tamil and English on every screen, with correct text rendering and keyboard entry.
- **Load:** simulate thousands of phones syncing in the same hour and check response times, error rates and database load.
- **Field pilot:** a real GN division before national rollout.

13. Risks and How We Keep It Simple

Risk	Mitigation
Sync bugs lose data	Never delete local records until the server confirms; keep "synced" copies for 30 days
Agents with low technical skill	Big buttons, clear sync badges, short training, Sinhala/Tamil UI
Poor internet for days	"Download my area" once; queue is unlimited
Scope creep	Freeze the field list in Phase 0; extra features go to a version 2 list
Privacy complaints	Consent screen, role guards, encryption, limited export rights
Android background limits	Always provide the manual Sync now button as backup
API or Supabase outage	Local data remains safe; sync resumes automatically
Generated login details shared or leaked	Random IDs and passwords, forced password change, rate limits, and an admin can reset or deactivate an account
Agents' own phones vary widely	Minimum is Android 10; test on low-end phones running it
Lost or shared phone	PIN lock, encrypted local database, immediate account deactivation

Risk	Mitigation
Direct APK is blocked or distrusted	Signed APK, checksum, one download page, install guide with screenshots, Google Play later
Bad data in Excel files	Row-by-row validation and a preview before anything is saved
Report layouts change	Keep report templates in one API module; agree layouts with the client in Phase 0

14. Open Items to Confirm

Confirmed

- The app is called the Global Research & Survey App (GRS); the Door-to-Door Skills Campaign is its first use.
- Role titles DC and SDA are used as written in the DD/S/002/2026 document; Level 1 is Admin CG.
- Dropdown lists for education, vocational streams and jobs are accepted; they live in a lookup table so they can change later.
- The house list is loaded by bulk Excel import and by typed entry; the Excel template columns are checked against the election name list layout.
- Agents use their own phones, Android 10 or newer.
- The dataset is accessed and owned by the super admin and admin Levels 0 to 4, and each level sees the records related to it.
- The app is distributed as a direct APK for now.
- Reports are this platform's own reports, not a government format.
- The Level 4 account acts as the team leader and chooses the houses for their team; each team has its own Level 4 leader only.
- Each team has its own member limit (for example 10 members in one team and 20 in another), set by Level 00 or Level 0; for now Level 00 sets it.
- Admin Levels 0 to 3 can also use the mobile app and enter data, and Levels 1 to 3 can create their own team members' accounts.
- Admins create accounts with a generated ID (GRS plus 10 digits) and a random password; users can change the ID to their email and change the password, and can log in with either.

- The NestJS API keeps no data of its own; records go to Supabase once synced. No deletion period is set for now.
- The NestJS API is hosted on Vercel.
- Expected agents: 50,000 to 1,000,000 is the long-term goal and the first launch is smaller; the busiest hours are most probably 10 a.m. to 3 p.m.

Open questions and checks

1. Level 0: its role has to be decided with the client.
2. Who creates which accounts: assumed that each level creates the level directly below plus its own team members (section 3.2); confirm with the client.
3. Build order: assumed Levels 00, 4 and 5 first, then Levels 1 to 3.
4. Scale: how many agents the first launch will have and how fast it grows toward the long-term goal, so database size and costs can be planned in steps. Records per agent per day depends on the campaign and is still unknown.
5. Vercel test deployment in Phase 1: confirm the region, Excel import and report generation work within Vercel's limits (see section 6).
6. The Excel template columns come from the format file (see section 4.3); check them against a real election name list file once the client provides one.

15. Summary

Build the Global Research & Survey App as a Flutter Android app with a local encrypted database and an outbox-style sync, a NestJS API on Vercel for login, sync, Excel import and reports, Supabase PostgreSQL as the central database, and a Flutter Web dashboard. Admins create accounts with generated IDs and passwords, and users can switch to email login. Plan for a long-term goal of 50,000 to 1,000,000 agents with the measures in section 6.1, and keep everything in Sinhala, English and Tamil. Keep the forms exactly as the paper versions, freeze the scope early, pilot in one division with a direct APK, load test before wide rollout, and extend to iOS afterward from the same code.